

EdgeCDSS 4.0

The Technology, Explained · Changes from Prior Versions · AI in Austere Medicine Project · July 2026

1. What EdgeCDSS is

EdgeCDSS is a clinical decision support system built for medics working in austere environments — prolonged field care, far-forward positions, disaster response — where specialty consultation is hours or days away and connectivity is fragile. A medic describes the casualty in ordinary field language, by text or voice, and the system responds in seconds with structured guidance: what to do, exact volumes to draw, what to watch, what never to give — cited to the Joint Trauma System Clinical Practice Guidelines (JTS CPGs) that govern military trauma care.

Version 4.0 is a milestone for two reasons. Architecturally, it completes the shift to a "deterministic-first" design in which Python code — not an AI model — owns every calculation and safety decision that can be computed. Operationally, it moves the entire system off rented cloud servers onto a small edge computer (an NVIDIA Jetson Orin Nano) owned and operated by the project, reachable worldwide through satellite internet. The system a medic queries today is a \$249 computer with the entire JTS library in local memory.

2. What changed since the last version

The v2.5/v3.x generation established the concept: a cloud-hosted pipeline pairing guideline retrieval with a language model, hardened by an expanding rulebook in its prompts. Version 4.0 is a structural rebuild of that concept:

- **Deterministic-first pipeline** — every dose the system states is computed by Python from confirmed patient weight and drug concentration, then handed to the language model as a fixed contract (ALLOWED_DOSES). The model formats language; it is prohibited from doing medication math. Thirteen deterministic pre-gates run before any AI is consulted — missing weight, missing route, pediatric limits, and known contraindications are decided in code, in milliseconds.
- **Structured patient context** — the system tracks a structured patient (confirmed vs. estimated weight, age, access, route preference) rebuilt from the entire conversation on every request. A weight confirmed early in a prolonged-care session holds for the whole encounter; age-estimated weight is walled off and never used for dosing.
- **Clinical router** — a routing index — built once by having an LLM read the entire knowledge base and extract each protocol's conditions, medications, and search vocabulary — maps field slang and clinical shorthand to the right CPG and sharpens retrieval before it happens.
- **Validator and fail-closed safety gate** — every draft response passes a second, narrow AI check (is anything unsafe?) followed by a deterministic gate. If validation fails, the medic sees a safety hold, never a corrected guess — the system fails closed. Structured overrides, written in code from observed false positives, keep the gate honest without weakening it.
- **Self-hosted edge deployment** — the full stack — retrieval database, embeddings, API server, web interface, audit logging — runs on the Jetson. The prior cloud deployment was retired after a provider capacity outage took it offline; v4.0 cannot be taken offline by a cloud vendor. Public access flows through an outbound-only encrypted tunnel: no open ports, no exposed addresses.
- **Structured clinical feedback and audit logging** — medics assess every response in the interface: one tap for "clinically appropriate," or a structured flag with severity (minor / significant / dangerous-if-followed), issue categories, and a protocol-cited correction. Every query is session-logged (no PHI) for audit and regression testing. A beta-reported false block was diagnosed from this log and fixed the same day.

- **Clinical speech normalization** — voice responses now speak clinical notation correctly — "0.24 mL of 100mg/mL" is pronounced "zero point two four milliliters of one hundred milligrams per milliliter," with routes and acronyms (IV, RSI, GCS) spelled out.
- **Test coverage** — an automated 24-case clinical suite — pediatric gates, P1 safety blocks, RSI protocols, scenario responses — passes 24/24 against the live public endpoint, up from 61.8% two versions ago.

3. The technology, in plain language

3.1 Retrieval-Augmented Generation (RAG)

A language model on its own answers from memory — fluent, but unverifiable and sometimes wrong. RAG changes the question from "what do you remember?" to "what do these documents say?" EdgeCDSS ingested all 89 JTS CPGs, splitting them into 8,559 passages ("chunks") along sentence boundaries, with page headers and footers stripped and each chunk tagged with its protocol, page, and date. Each chunk is converted to an embedding — a numerical fingerprint of meaning — and stored in a vector database (ChromaDB) on the Jetson itself.

When a query arrives, the system embeds it the same way and retrieves the passages whose meaning sits closest. Those passages, with their citations, are what the language model is given to write from. Retrieval scores are graded honestly: high-confidence matches are labeled as JTS-grounded; weak matches force the response to be labeled general medical knowledge rather than dressed up as protocol. Because embeddings are computed on-device, retrieval costs nothing per query and keeps working when connectivity degrades.

3.2 Prompt engineering

Prompt engineering is the discipline of writing the instructions a language model operates under. In EdgeCDSS these prompts are not suggestions — they are contracts. The generation prompt receives the patient context, the retrieved guideline text, and the ALLOWED_DOSES block, and is instructed to present only those doses, in a fixed response format (DO THIS / GIVE / WATCH / DON'T / TLDR / SOURCE) designed to be read under stress. The validation prompt is deliberately narrow: it lists the exact failure classes to check — pediatric dosing without confirmed weight, contraindicated drugs, missed critical diagnoses — and explicitly forbids inventing concerns beyond that list. Version 4.0's lesson is that prompts alone are not enough: every prompt rule that matters is backed by a deterministic check in code.

3.3 The deterministic-first pipeline

The pipeline's ordering principle: never ask an AI a question that code can answer. A query first passes the thirteen pre-gates (weight known? route known? pediatric limits? absolute contraindications?) — many queries are fully answered here in under five milliseconds without any AI involvement. Only then does retrieval run, the router enhance the search, and the language model write — inside its dose contract. The draft then faces deterministic post-checks (does every number match the contract?) and the validator. The medic sees the result only if every stage passes; otherwise the system holds and says so. Each stage is separately loggable, testable, and fixable — which is what turns beta feedback into same-day, surgical fixes.

3.4 Edge deployment and connectivity

Everything except language generation runs on the Jetson: an eight-gigabyte ARM computer drawing under 25 watts, running the API server, vector database, embeddings, audit log, and web interface as system services that survive power cycles. Global reach comes from pairing satellite internet (Starlink) with an outbound-only Cloudflare Tunnel: the Jetson dials out and holds the connection; queries flow down through it. Nothing on the network accepts inbound connections, no ports are forwarded, and the device's address is never exposed. The recurring cost of the

entire system is dominated not by computing or AI but by connectivity — roughly \$120–150 per month of satellite service, alongside single-digit dollars of language-model API use.

4. Why this architecture can be trusted — and its limits

Trust here does not mean trusting an AI. It means the load-bearing elements are inspectable: dose math is code that can be read and tested; safety gates are code; the knowledge base is the JTS corpus itself, cited by protocol and page in every grounded response; and the system’s failure mode is refusal, not improvisation. The 24-case suite runs against the live endpoint after every change.

The limits are equally explicit. EdgeCDSS is decision support, not a decision-maker — guideline-derived assistance that presumes a trained provider applying clinical judgment and local protocol. Language generation still requires connectivity to a cloud model (a fully offline on-device model is a research goal, and the generation layer is deliberately swappable — a Claude/Gemini comparison study is planned). Retrieval quality is bounded by the corpus: it reflects the JTS CPGs as published, no more and no newer. And the beta is the mechanism by which the remaining unknown failure modes are found — which is precisely why structured medic feedback is built into every response.